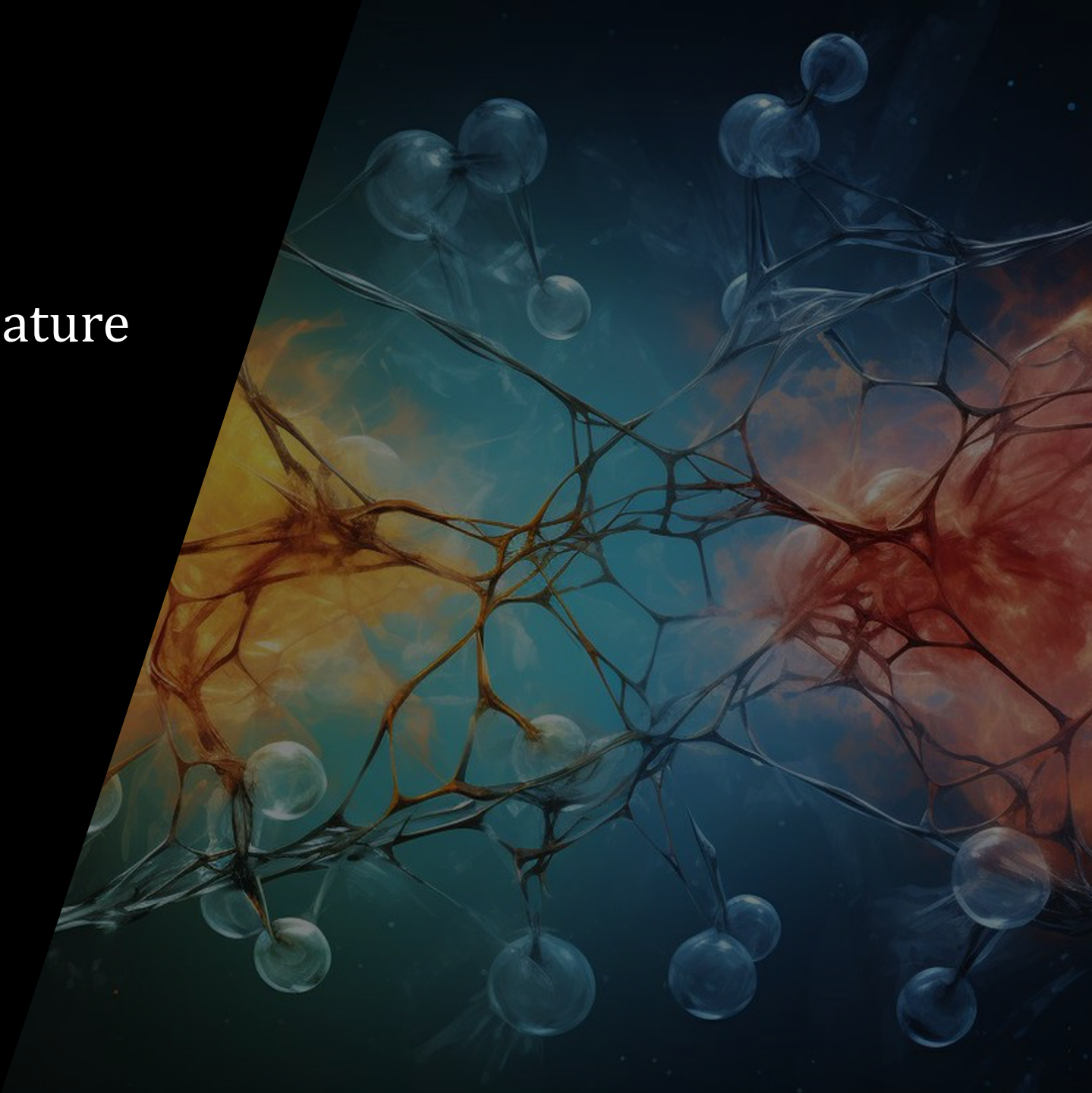


Session 3 - Linear models on feature vectors

Contents:

- *Linear regression model*
- *L2-loss for regression and normal equations*
- *Linear classification model*
- *Softmax function, cross-entropy loss for classification*

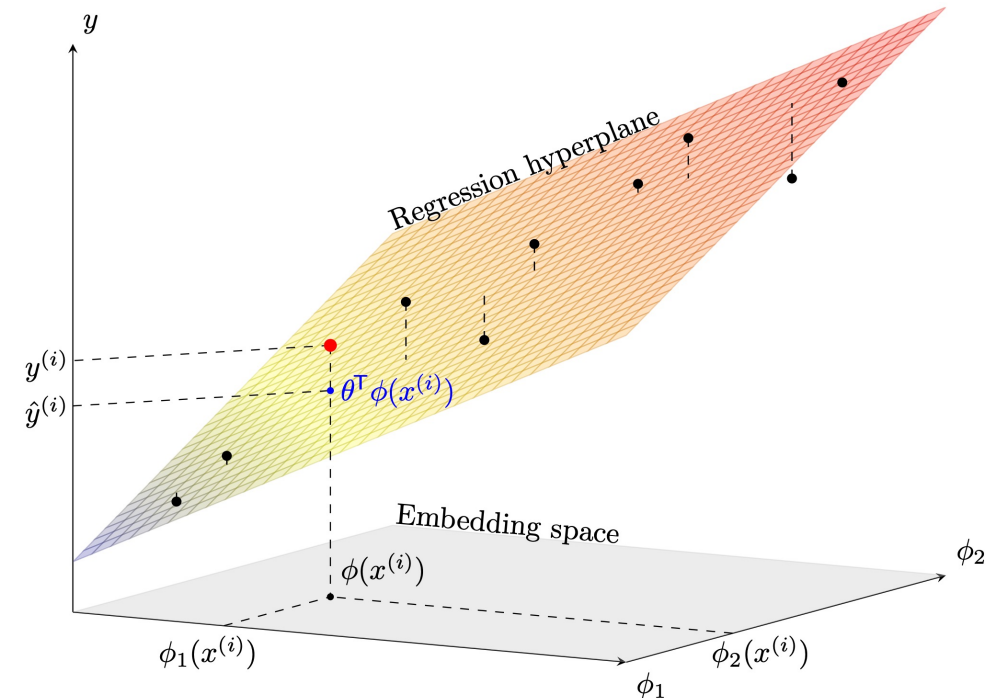


Linear regression

- What kind of problems one can solve efficiently, even in large dimensions? → **Linear systems!**
- Let us talk first about regression: the answer is modelled as

$$f_{\theta}(\phi_1^{(i)}, \phi_2^{(i)}, \dots) = \theta^T \phi^{(i)} = \hat{y}^{(i)}$$

- It is sometimes convenient to add an affine term (also called **bias** in the neural network literature), which can be absorbed in the feature vector making it of dimension $d' + 1$ where $\theta = [\theta_0, \theta_1, \theta_2, \dots]^T$, $\phi^{(i)} = [1, \phi_1^{(i)}, \phi_2^{(i)}, \dots]^T$.
- **Geometric interpretation:** projection of an embedding vector onto a **hyperplane** parameterised by θ .



Linear regression

- Example: salary prediction based on the years of experience
- Data are 373 couples $(\phi^{(i)}, y^{(i)}) \Rightarrow$ **Supervised learning**
- The target variable $y \in \mathbb{R}$ is continuous $\Rightarrow$ **Regression**
- The linear model is

$$\hat{y}^{(i)} = \theta_0 + \theta_1 \phi_1^{(i)},$$

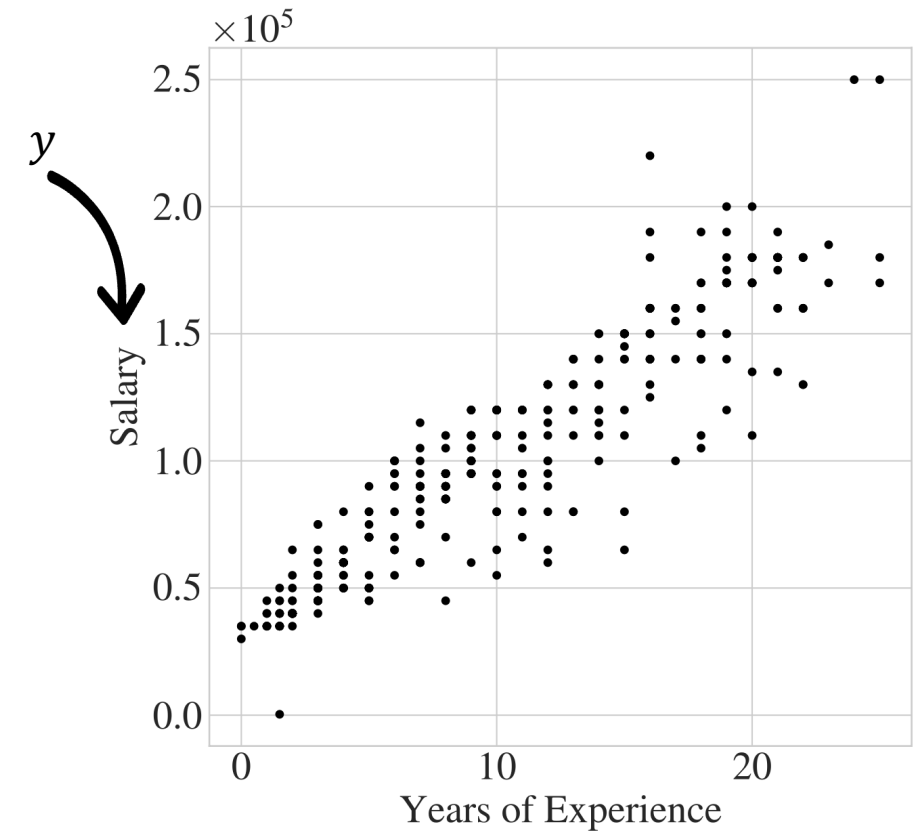
where $\phi_1^{(i)}$ is the nb. of years of experience of the i^{th} training example

- Now the model is fixed, how to find $\hat{\theta}$, the best possible parameters for our model and data?
- This is done using **empirical risk minimisation (ERM)**

$$\hat{\theta} = \underset{\theta}{\operatorname{argmin}} R(\mathbf{X}, \theta) = \underset{\theta}{\operatorname{argmin}} \frac{1}{n} \sum_{i=1}^n \ell(\hat{y}^{(i)}, y^{(i)})$$

Cost (or error) function: measures the error of the model on the training set

Loss function: measures the error of the model on an individual point



Linear regression

- A common **choice** of loss for regression is a **squared loss function**

$$\hat{\boldsymbol{\theta}} = \operatorname{argmin}_{\boldsymbol{\theta}} \frac{1}{n} \sum_{i=1}^n (\hat{y}^{(i)} - y^{(i)})^2$$

- Here**, the optimisation problem can be solved analytically in closed-form. Rewriting the risk matrixially, we have

$$R(\mathbf{X}, \boldsymbol{\theta}) = \frac{1}{n} \|\boldsymbol{\Phi}\boldsymbol{\theta} - \mathbf{y}\|_2^2$$

Feature matrix

$$\boldsymbol{\Phi} = \begin{pmatrix} \phi_1^{(1)} & \dots & \phi_{d'}^{(1)} \\ \vdots & \ddots & \vdots \\ \phi_1^{(n)} & \dots & \phi_{d'}^{(n)} \end{pmatrix} \in \mathbb{R}^{n \times d'}$$

Target vector

$$\mathbf{y} = [y_1, \dots, y_n]^T \in \mathbb{R}^n$$



The analytical minimisation of the squared loss in linear regression gives the unique solution (when $d' < n$) known as **normal equations**

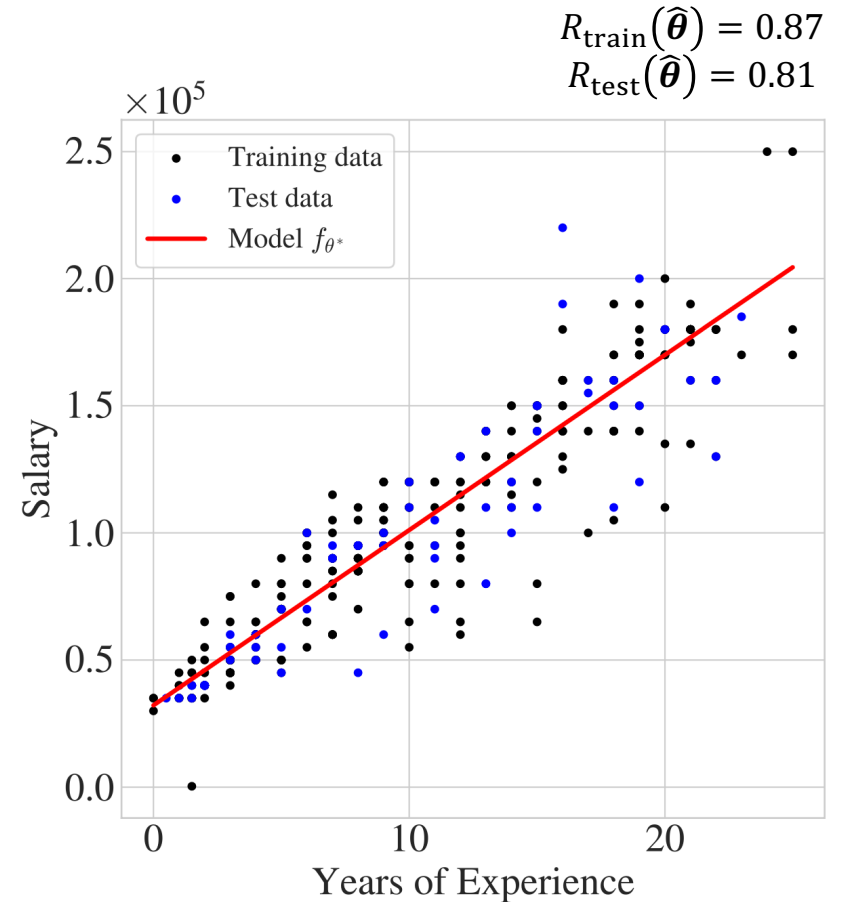
$$\hat{\boldsymbol{\theta}} = (\boldsymbol{\Phi}^T \boldsymbol{\Phi})^{-1} \boldsymbol{\Phi}^T \mathbf{y}$$

Linear regression



1. I **first** separated the dataset into **training and test sets**, $n = 298$ and $n_{\text{test}} = 75$ chosen randomly.
2. Then, I computed the optimal parameters minimising the empirical risk using the normal equations on the training features
$$\hat{\theta} = (\Phi^T \Phi)^{-1} \Phi^T y.$$
3. I computed the risk on the train and test sets and found they are close.

- + Exactly solvable model, low variance
- Cannot represent local relationships, may be biased



Linear classification

- Consider a K -class classification problem for which features ϕ allow linear separability
- Example: Iris dataset with 150 couples $(\phi^{(i)}, y^{(i)}) \Rightarrow$

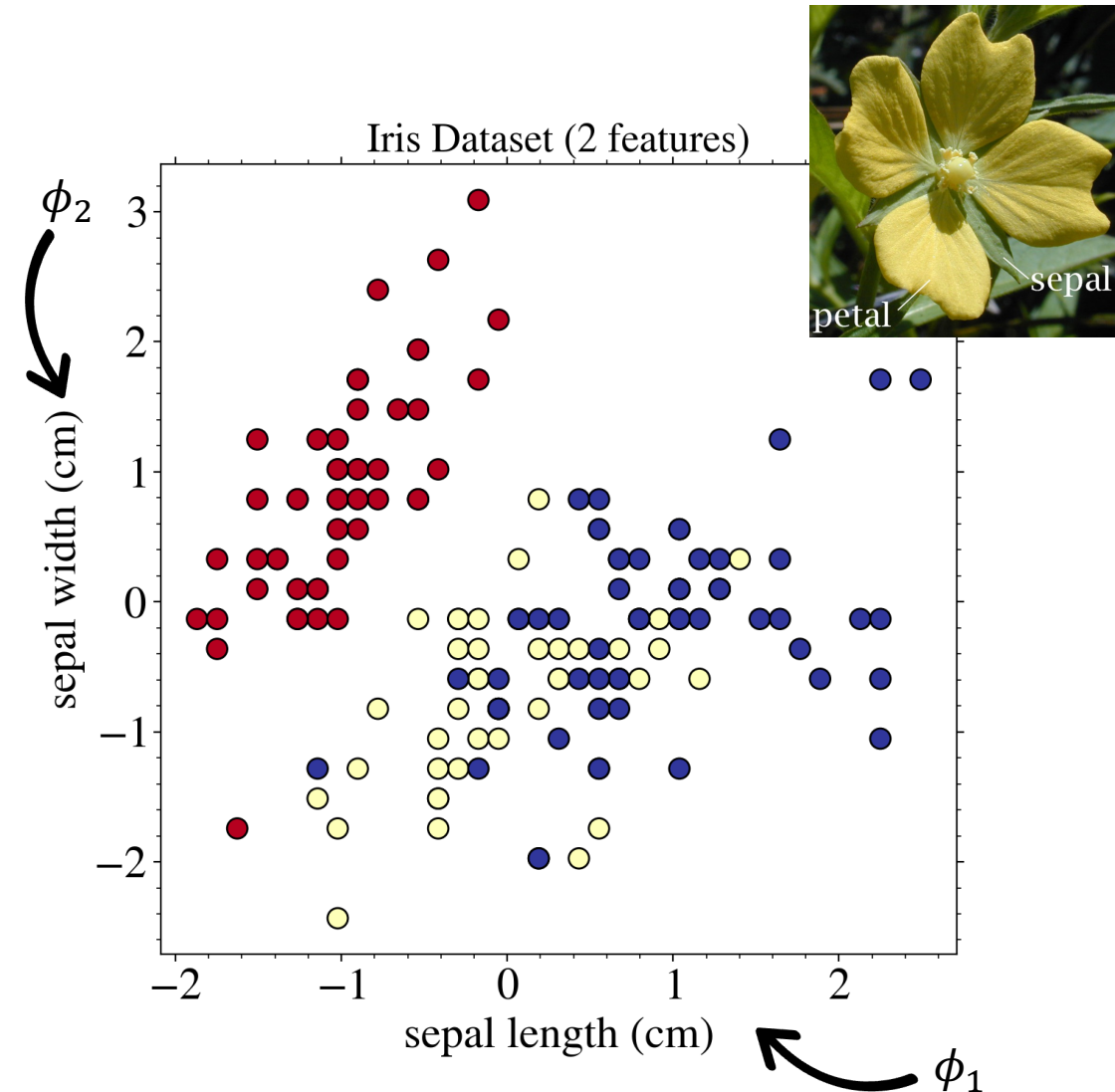
Supervised learning

- The target variable $y \in \{0,1,2\} \Rightarrow$ **Classification**
- A natural loss function for classification is the **0-1 loss**

$$\ell(y, \hat{y}) = \begin{cases} 1 & \text{if } \hat{y}^{(i)} \neq y^{(i)}, \\ 0 & \text{otherwise.} \end{cases}$$

- The optimal decision rule (in Bayes sense) is

$$\hat{y} = \operatorname{argmax}_k p(y = k | \phi).$$



We thus need a **model** $p_{\theta}(y = k | \phi)$ of the conditional probability distribution to perform classification!

Linear classification

- The simplest models assume a linear log probability

$$\log p_{\theta}(y^{(i)} = k | \phi^{(i)}) = \theta_k^T \phi^{(i)} - \log Z$$

where Z is a normalizing constant so that probabilities sum to one.

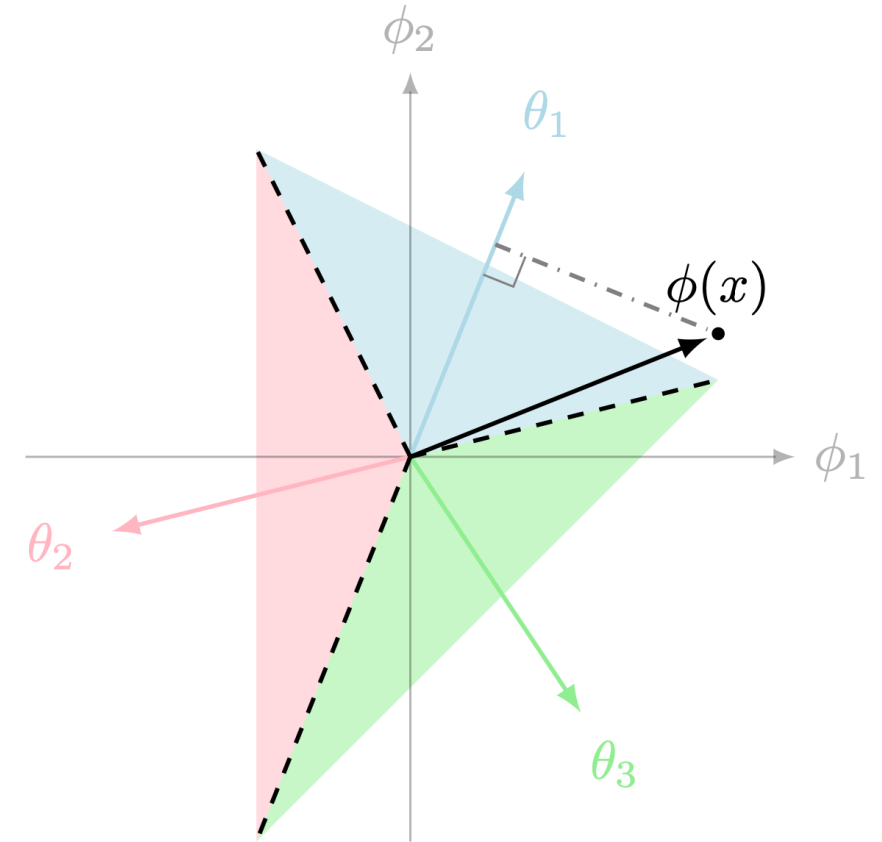
- It means that

$$p_{\theta}(y^{(i)} = k | \phi^{(i)}) = \frac{\exp(\theta_k^T \phi^{(i)})}{\sum_{j=1}^K \exp(\theta_j^T \phi^{(i)})}$$

which is called the **softmax function** allowing to turn the linear responses for each class into probabilities.

- And the classification rule is

$$\hat{y} = \operatorname{argmax}_k \theta_k^T \phi^{(i)}$$



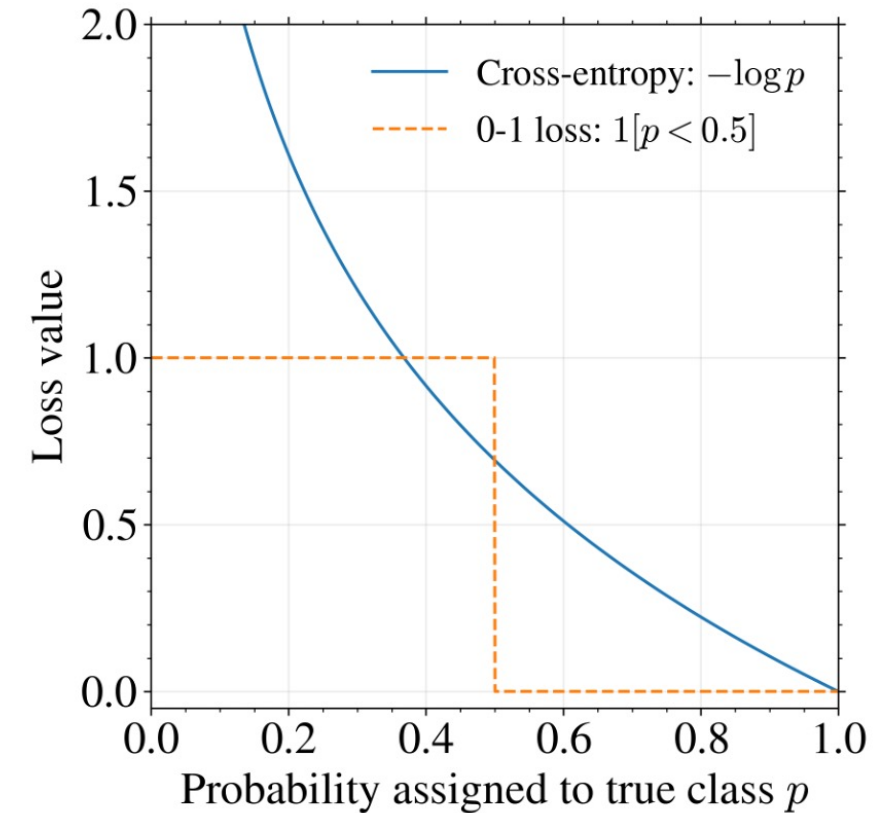
Geometrically, it corresponds to computing the overlap between the feature $\phi^{(i)}$ and a vector representative for each class, θ_k , and associating **the class maximising the dot product**, leading to **linear decision** boundaries shown as hyperplanes.

Linear classification

- Now the model is specified, we need minimise the risk to obtain the parameters θ_k using some training data
- For optimisation, the 0-1 loss is not suitable since it is not differentiable, but we can relax it using the probabilities

$$\ell(y, \hat{y}) = - \sum_{k=1}^K 1_{y=k} \log p_{\theta}(y = k | \phi^{(i)})$$

which is **now differentiable and convex**.

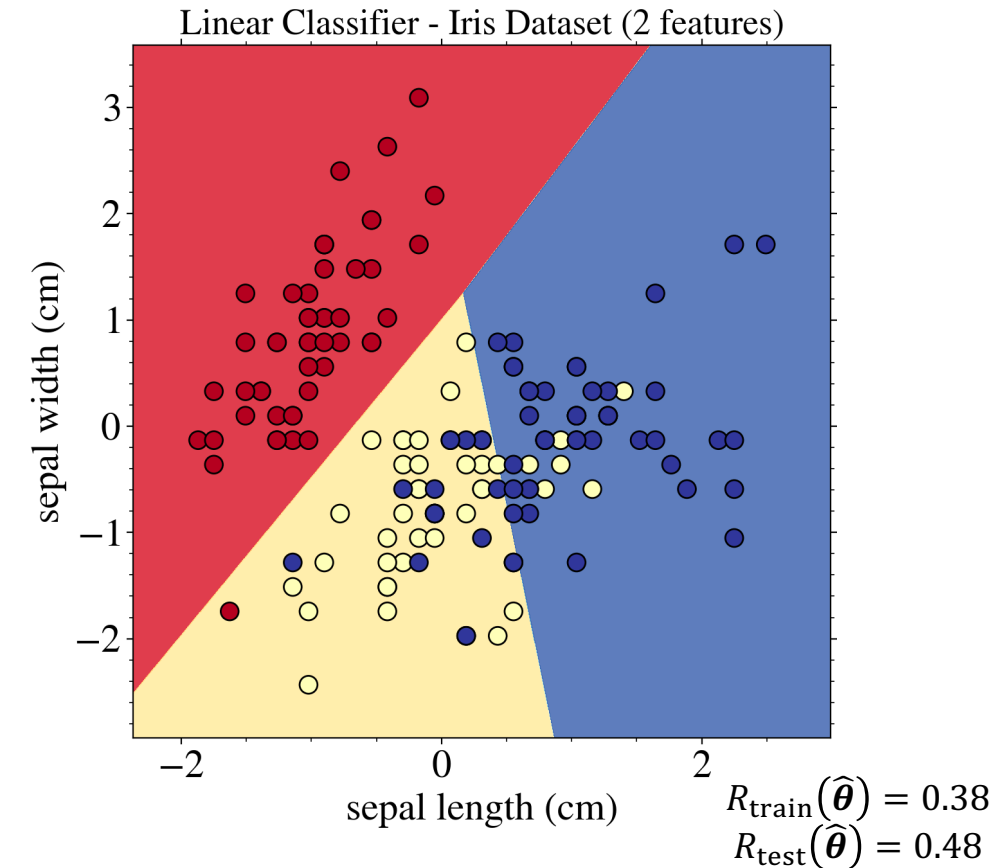


Linear classification

- Now the model is specified, we need minimise the risk to obtain the parameters θ_k using some training data
- For optimisation, the 0-1 loss is not suitable since it is not differentiable, but we can relax it using the probabilities

$$\ell(y, \hat{y}) = - \sum_{k=1}^K 1_{y=k} \log p_{\theta}(y = k | \phi^{(i)})$$

which is **now differentiable and convex**.



This error function is referred to as **cross-entropy** and it is the most widely used cost function for classification problems. The parameters of the model are then obtained by minimising the empirical risk over the training set, i.e.

$$\hat{\theta} = \operatorname{argmin}_{\theta} \hat{R}(\mathbf{X}, \theta) = \operatorname{argmin}_{\theta} \frac{1}{n} \sum_{i=1}^n \ell(y^{(i)}, \hat{y}^{(i)}).$$